

Application Architectures

Software Engineering — Ian Sommerville (10th Edition)

Chapter 6: Architectural Design — Part 3 Study Notes

These notes cover all 11 topics from the Application Architectures section, discussed topic by topic based on Sommerville's slides.

Topic 1: Why Application Architectures are Needed

The Core Idea

Think about this — every phone company in the world needs to connect calls, meter call duration, manage their network, and send bills to customers. Should every phone company design their software from scratch? That would be wasteful. Instead, they share a common underlying architecture and customize it for their brand, region, or pricing model. This is exactly what a Generic Application Architecture is.

A generic application architecture is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.

What is a Generic Application Architecture?

It is basically a template architecture that captures the principal characteristics of a class of systems. Examples:

- All e-commerce systems (Amazon, Daraz, eBay) share a common architecture — product catalog, cart, payment, order tracking
- All banking systems share a common architecture — accounts, transactions, authentication, reporting
- All real-time systems share common architectural models

Why is this Useful? (3 Uses from Sommerville)

- **As a starting point for design:** Use the generic architecture as your base and build on top of it instead of designing from scratch.
- **As a checklist:** Compare your designed architecture against the generic one to ensure nothing important is missed.
- **As a way to organize the development team:** Assign teams to different parts of the architecture independently.

Real World Example

A Supply Chain Management system — the same generic architecture can be adapted for different suppliers, different types of goods, and different contract terms. Same skeleton, different skin.

Topic 2: Types of Application Systems

The Core Idea

Sommerville categorizes applications into 4 broad types based on how they process data and interact with users. Think of it like vehicles — a bicycle, car, truck, and airplane all transport you, but they work completely differently. Similarly, all software processes data but in very different ways.

1. Data Processing Applications

- Process data in batches with no user interaction during processing
- User gives input → system runs → output produced
- Like a factory assembly line — raw material in one end, finished product out the other
- **Example:** Payroll systems — processes all employee records at month-end automatically

2. Transaction Processing Applications

- Data-centered — revolve around a database
- Process user requests and update information in real time
- Must ensure consistency — all operations complete or nothing changes (atomicity)
- **Example:** ATM systems, online banking, e-commerce checkouts, booking systems

3. Event Processing Systems

- System actions depend on interpreting events from the environment
- The system listens for something to happen, then reacts
- **Example:** Fire alarm systems, stock trading systems, smart home systems

4. Language Processing Systems

- Users express intentions in a formal language
- System processes and interprets that language
- **Example:** Compilers, SQL engines, command line interpreters

Type	User Interaction	Core Activity	Example
Data Processing	Minimal/None	Batch processing	Payroll system
Transaction Processing	High	DB read/write	ATM, e-commerce
Event Processing	Indirect	React to events	Fire alarm, stock trading
Language Processing	Via formal language	Translate/interpret	Compiler, SQL engine

Topic 3: Generic Application Architecture Examples

Out of the 4 types, Sommerville specifically zooms into 2 as the most widely used generic application architectures: Transaction Processing Systems and Language Processing Systems.

1. Transaction Processing Systems (Web-based business systems)

Examples: Interactive banking systems, E-commerce systems, Information systems, Booking systems. Every one of these systems — whether a bank or airline — needs the same core structure: UI for user interaction, validation and authentication, business logic to process requests, and a database.

2. Language Processing Systems (Software development)

These systems take input written in a formal language and process it. Examples: Compilers (HLL to machine code), SQL interpreters, Command interpreters. Every compiler follows the same architectural steps: read source code, analyze it, generate intermediate representation, and produce output.

	Transaction Processing	Language Processing
Used in	Business & web systems	Software development tools
Core activity	DB read/write operations	Translate/interpret formal language
Architecture style	Layered + Client-Server	Pipe & Filter + Repository
Common example	ATM, e-commerce	Compiler, SQL engine

You don't need to design every system from scratch. Recognize what type of system you're building, find the generic architecture for that type, and adapt it.

Topic 4: Transaction Processing Systems

What Exactly is a Transaction?

A transaction is a sequence of operations treated as a single atomic unit. Atomic means All or Nothing — either every single operation completes successfully, OR nothing is saved at all. There is no in-between.

Classic Example — Bank Transfer: You transfer Rs. 5000 from Account A to Account B. If the system crashes after deducting from A but before adding to B — without atomicity, the money vanishes. With atomicity, the system rolls back and Account A is restored.

Two Perspectives of a Transaction

- **System Perspective:** A strict sequence of operations that must all complete before any database changes are made permanent.
- **User Perspective:** Simply any coherent sequence of operations that satisfies a goal. Example: 'Find flights from Karachi to Dubai' — simple to user, multiple operations to system.

How Requests are Handled

Users make asynchronous requests for service processed by a Transaction Manager. Asynchronous means the user doesn't wait doing nothing — like ordering food at a restaurant: you place the order, talk to friends, and food arrives when ready.

System	What user does	What transaction does
ATM	Withdraw cash	Verify PIN → check balance → deduct → dispense
E-commerce	Place an order	Verify payment → update inventory → create order
Flight booking	Book a seat	Check availability → reserve → process payment
Online banking	Check balance	Authenticate → query account → return balance

Topic 5: Structure of Transaction Processing Applications

The 4 Main Components

1. I/O Component (Input/Output)

The user-facing part of the system. Handles how the user sends requests and receives responses. Could be a web form, ATM keypad, or mobile app screen.

2. Application Logic

The brain of the system. Contains business rules specific to the application. Validates input and decides what to do with the request. Sits between the user and the transaction manager.

3. Transaction Manager

The controller of database operations. Receives the transaction from application logic. Ensures atomicity — manages commit and rollback. Keeps the database consistent.

4. Database

Where all data is permanently stored. Only updated after the transaction manager confirms everything went well.

The Flow — Step by Step

- Step 1: User makes a request using the I/O component
- Step 2: Request is processed by application-specific logic
- Step 3: A transaction is created and passed to the Transaction Manager
- Step 4: Transaction Manager processes it against the database
- Step 5: After successful completion, confirmation message is passed back to application logic
- Step 6: Application logic sends the response back to the user via I/O component

If anything fails between Step 3 and Step 5 — the Transaction Manager rolls back everything. The database returns to its previous state. No partial updates. No data corruption. This is the transaction manager's most critical job.

ATM Mapped to Components

Component	In ATM Context
I/O Component	Keypad, card reader, screen, cash dispenser
Application Logic	Validates PIN, checks account type, applies withdrawal rules
Transaction Manager	Handles deduction from account atomically
Database	Bank's central database with all account records

Topic 6: ATM System as Pipe & Filter Architecture

The Core Idea

Sommerville takes the ATM system (a Transaction Processing System) and shows how it can also be modeled using the Pipe & Filter architectural pattern. This teaches something very important:

A single system can be viewed through multiple architectural lenses. The pattern you choose depends on what aspect you want to highlight.

The 2 Cooperating Software Components

- **ATM Software:** Installed physically inside the ATM machine. Handles everything the user directly interacts with.
- **Bank Processing Logic:** Runs on the Bank's database server (remotely). Handles the actual financial processing.

The 3 Filters in the ATM Pipeline

Filter 1: Input (ATM Side)

Card reading, PIN capture, user menu selection, amount entry. Transforms physical user actions into a digital transaction request.

Filter 2: Process (Bank Server Side)

PIN verification, account validation, balance checking, transaction execution. Transforms transaction request into a transaction result.

Filter 3: Output (ATM Side)

Cash dispensing, receipt printing, screen confirmation, card ejection. Transforms transaction result into physical output to user.

Why Model it as Pipe & Filter?

Model	What it Highlights
TPS Model (Topic 5)	Data consistency, atomicity, transaction safety
Pipe & Filter Model (Topic 6)	Sequential flow of processing stages, separation of concerns

The Physical Split — Key Insight

The ATM machine handles Filters 1 and 3 (Input & Output) while the bank's server handles Filter 2 (Processing). They communicate over a network. This is why ATMs can work across different banks — the processing filter at your bank's server is separate from the physical ATM machine.

Topic 7: Information Systems Architecture

The Core Idea

Information Systems (IS) are database-centered systems that allow controlled access to a large base of information. Examples: Hospital patient record systems (Mentcare), Library management systems, University student portals, HR management systems, Airline reservation systems.

The 4 Layers

Layer 1 (Top): User Interface

What the user directly sees and interacts with. Web browser, mobile app, desktop application. Presents data in a readable format and sends user requests downward.

Layer 2: User Communications, Authentication & Authorization

Manages who is allowed to do what. Handles login, session management, access control. Example: A doctor can view AND edit patient records; a receptionist can only view them.

Layer 3: Information Retrieval (Application Logic)

The brain of the system. Contains business logic for accessing and updating data. Processes validated requests and decides exactly what to fetch or update.

Layer 4 (Bottom): System Database

Where all data is permanently stored. Only accessed through Layer 3 — never directly by users. Protected behind all the layers above it.

Physical Deployment — 3-Tier Architecture

The 4 logical layers map to 3 physical servers:

Physical Server (Tier)	Logical Layers it Handles
Web Server (Tier 1)	Layer 1 — User Interface
Application Server (Tier 2)	Layers 2 & 3 — Auth + Business Logic
Database Server (Tier 3)	Layer 4 — System Database

Important: Do NOT confuse Layers (logical separation of responsibilities in design) with Tiers (physical separation of machines/servers in deployment). Information Systems have 4 logical layers but are deployed across 3 physical tiers.

Mentcare System Example (Sommerville's Case Study)

Layer	In Mentcare
User Interface	Browser-based UI
Auth & Authorization	Login system, role-based access (doctor vs nurse vs admin)

Application Logic	Patient records, appointment scheduling, prescription management
Database	All patient records, appointments, medical histories

Topic 8: Language Processing Systems

The Core Idea

A Language Processing System (LPS) is a system that takes input written in a formal language and translates or interprets it into another form that can be executed or understood by a machine.

Sommerville: 'Systems that translate texts from one language into another and carry out the instructions specified in the input language.'

What is a Formal Language?

A formal language has strict, precise rules — not natural language like English where meaning can be ambiguous. Examples: Programming languages (Python, Java, C++), Query languages (SQL), Markup languages (HTML, XML), Command languages (Bash, PowerShell). One missing semicolon and the whole thing fails — that strictness is what makes them formal.

The Two Core Components

- **Translator:** Takes the source language as input, converts it into another form (machine code, bytecode, or intermediate representation).
- **Abstract Machine:** Executes the translated output. Actually runs the instructions produced by the translator.

Real World Examples

Example	Formal Language Input	Translator	Abstract Machine
Compiler	Python/Java code	Compiler	CPU / JVM
SQL Engine	SQL Query	SQL Parser	Database Engine
Terminal	ls -la command	Shell Parser	OS Kernel

Architectural Patterns in LPS

Pattern	Role in LPS
Pipe & Filter	Each stage of translation is a filter, data flows through sequentially
Repository	Shared data (symbol tables, parse trees) stored centrally, accessed by multiple components

Topic 9: Compiler Components

The Core Idea

A compiler is the most complete and classic example of a Language Processing System. It takes high-level source code (Python, Java, C++) and transforms it into machine code that a CPU can execute through multiple specialized components.

Component 1: Lexical Analyzer (Lexer/Scanner/Tokenizer)

Reads raw source code character by character, groups characters into meaningful units called tokens, and removes whitespace and comments.

- Input: `total = price + tax`
- Output tokens: `total (Identifier) | = (Operator) | price (Identifier) | + (Operator) | tax (Identifier)`

The lexer doesn't care about meaning yet — it just identifies and categorizes the pieces.

Component 2: Symbol Table (Shared Repository)

Stores information about all identifiers in the program — variables, functions, classes, their types, scope, memory location. Shared and accessed by multiple components throughout compilation. This is the Repository pattern at work!

Component 3: Syntax Analyzer (Parser)

Takes the stream of tokens from the Lexer, checks if they follow the grammatical rules of the language, and builds a Parse Tree (Abstract Syntax Tree).

- **Valid syntax:** `total = price + tax`
- **Invalid syntax:** `total = + price tax` (syntax error)

The parser doesn't check meaning yet — just structure.

Component 4: Semantic Analyzer

Takes the parse tree and checks if the code actually makes sense — verifies types, scope, and logical consistency.

Error Type	Example	Caught By
Syntax Error	<code>total = + price tax</code>	Syntax Analyzer
Semantic Error	<code>total = "hello" + 5</code>	Semantic Analyzer

Component 5: Code Generator

Takes the verified, analyzed representation and generates the target output — machine code, bytecode, or assembly. Also optimizes code for performance where possible.

Connecting to Architectural Patterns

Component	Architectural Pattern
Lexer → Parser → Semantic Analyzer → Code Generator	Pipe & Filter — sequential stages, each transforms data
Symbol Table	Repository — shared data store accessed by multiple components

Topic 10: Repository Architecture for Language Processing Systems

The Core Idea

Sommerville shows how a complete Language Processing System — an entire Integrated Programming Environment (IDE) like VS Code, Eclipse, or IntelliJ — is organized using the Repository architectural pattern. All tools share one central repository.

The Problem Without a Repository

If each tool maintains its own separate copy of code information:

- **Duplication:** Same data stored multiple times
- **Inconsistency:** One tool's data gets updated, others don't know
- **Tight coupling:** Tools need to constantly communicate with each other

What's Stored in the Repository?

Stored Item	What it is
Source Code	The actual code files being worked on
Parse Trees	Syntactic structure of the code
Symbol Table	All identifiers, types, scopes
Grammar Definition	Rules of the programming language
Output Definition	Rules for generating target code

Note: Grammar definition and output definition might sometimes be embedded in tools rather than stored in the repository.

Advantages in this Context

- **Tool Independence:** Each tool can be developed, updated, or replaced without affecting other tools.
- **Consistency:** All tools always work with the same version of the code representation.
- **Easy to Add New Tools:** Just connect a new tool to the repository — no need to integrate with every existing tool.

Real World — IntelliJ IDEA

When you type code in IntelliJ: Editor writes source code → Lexer tokenizes → Parser builds parse tree → Syntax highlighter colors code → Error checker shows red underlines → Autocomplete suggests names → Debugger maps errors to source lines. All happening simultaneously, all using the same central repository, all independent of each other.

Topic 11: Pipe & Filter + Repository — Composite Architecture

The Core Idea

This is the final and most complete picture Sommerville gives us. He combines Pipe & Filter and Repository into one unified architectural model for a complete Language Processing System.

Real systems are never just one pattern — they are intelligent combinations of multiple patterns working together.

The Problem with Pure Pipe & Filter

Each filter only talks to the next filter in line. If the Code Generator needs information discovered back in the Lexer stage, it must pass through every intermediate filter — even ones that don't need it. This creates unnecessary coupling. ■

The Problem with Pure Repository

No defined order of processing. The repository doesn't enforce sequence. Tools might try to read data that hasn't been written yet. The Code Generator might try to read the parse tree before the Parser finishes building it. ■

The Composite Solution

- **Pipe & Filter** enforces the processing order (sequence)
- **Repository** enables shared data access across all stages

Step by Step — How it Works

Step	Component	Reads from Repository	Writes to Repository
1	Lexer	Source code	Tokens, symbol table entries
2	Parser	Grammar rules	Parse tree
3	Semantic Analyzer	Parse tree, symbol table	Semantic annotations, updated symbol table
4	Code Generator	Parse tree, symbol table, output definitions	Final machine code

Why This Combination is Powerful

Challenge	Solved By
Need strict processing order	Pipe & Filter enforces sequence
Non-adjacent components need same data	Repository provides shared access
Adding a new processing stage	Add a new filter to the pipe
Adding a new tool (debugger, formatter)	Connect it to the repository

Complete Summary — All 11 Topics

Topic	Key Point
1. Why App Architectures	Generic architectures save time — adapt, don't rebuild
2. Types of Applications	Data Processing, Transaction, Event, Language Processing
3. Generic Examples	TPS and LPS are the two most widely used
4. Transaction Processing	Atomic operations, consistency, transaction manager
5. TPS Structure	I/O → App Logic → Transaction Manager → Database
6. ATM as Pipe & Filter	Same system, different lens — highlights data flow
7. Information Systems	4 layers, deployed as 3-tier client-server
8. Language Processing	Translator + Abstract Machine, formal language input
9. Compiler Components	Lexer → Parser → Semantic Analyzer → Code Generator + Symbol Table
10. Repository for LPS	All IDE tools share central repository independently
11. Composite Architecture	Pipe & Filter + Repository combined for complete LPS

The Master Lesson

No real system uses just one architectural pattern. Real systems are intelligent combinations of patterns, chosen based on what each part of the system needs to accomplish. The ATM combines Transaction Processing + Pipe & Filter. Information Systems combine Layered + Client-Server. Compilers/IDEs combine Pipe & Filter + Repository.